

metrics_scrape_test.sh — control-API Prometheus / metrics live-scrape proof

Revision: 1 **Last modified:** 2026-07-01T09:45:00Z **Authority:** Constitution §11.4.115 (RED-baseline polarity), §11.4.107 (liveness content), §11.4.69 (positive evidence), §11.4.119 (single live-boot owner), §11.4.18 (script documentation), §11.4.6 (no-guessing). **Scope:** observability wiring for the control-plane REST API's Prometheus metrics endpoint.

Overview

tests/observability/metrics_scrape_test.sh is the RED/GREEN polarity guard for the control-API's Prometheus /metrics endpoint. The control-plane REST API (control-plane/cmd/api) exposes an OPTIONAL separate plaintext /metrics listener enabled by the CONTROL_API_METRICS_ADDR environment variable; the listener is wired into the compose stack by docker-compose.observability.yml as the proxy-api service.

The script has two polarities selected by the RED_MODE environment variable:

- **RED_MODE=1** reproduces the "metrics not exposed" defect. It scrapes the endpoint and asserts NO valid Prometheus exposition is served (connection refused, or a body with no helix_proxy_metrics). This is provable BEFORE any boot and is the §11.4.115 RED baseline.
- **RED_MODE=0** (default) is the standing GREEN guard the conductor runs AFTER booting proxy-api. It scrapes /metrics and asserts REAL Prometheus exposition CONTENT — a known helix_proxy_metric name present, # HELP / # TYPE lines present, the acl-decisions counter parseable — and attempts the counter-increment-after-a-proxied-request proof. It asserts the scraped metric CONTENT, never merely HTTP 200 (§11.4 / §11.4.69 / §11.4.107).

The script performs NO boot. The live boot + scrape is owned by the conductor (§11.4.119 — exactly one stream owns each exclusive live resource). With defaults and nothing booted, the GREEN guard emits an honest SKIP, never a fake PASS.

Metric names asserted

Pinned to control-plane/internal/api/metrics.go:35-37:

Metric	Kind	Notes
helix_proxy_vpn_up{profile}	gauge	per profile, fail-closed from Redis; absent when no profiles exist
helix_proxy_acl_decisions_total{decision}	counter	decision=OK ERR, both pre-touched to 0
helix_proxy_tunnel_down_responses_total	counter	fail-closed 503 responses

The two counters are always present (pre-touched at registration), so they are the hard-required content. `helix_proxy_vpn_up` is present only when at least one VPN profile exists; its absence with zero profiles is recorded honestly, not failed (§11.4.6).

Prerequisites

- POSIX `sh`, `awk`, `grep`, `curl`.
- The canonical evidence helper `tests/lib/evidence.sh` (sourced for `ab_pass_with_evidence` / `ab_skip_with_reason`).
- For the GREEN live proof only: the conductor has booted `proxy-api` via the observability overlay (see the command sequence below), the operator has provisioned the four Podman secrets, and the control-plane image is built.

Usage examples

RED baseline (provable now, nothing booted):

```
RED_MODE=1 GOMAXPROCS=2 nice -n 19 ionice -c 3 \  
  bash tests/observability/metrics_scrape_test.sh
```

GREEN guard, authored-not-booted default (honest SKIP):

```
GOMAXPROCS=2 nice -n 19 ionice -c 3 \  
  bash tests/observability/metrics_scrape_test.sh
```

GREEN guard, live (conductor, after boot):

```
HELIX_OBSERVABILITY_STACK=1 \  
HELIX_METRICS_URL=http://127.0.0.1:34090/metrics \  
HELIX_PROXY_URL=http://127.0.0.1:34128 \  
GOMAXPROCS=2 nice -n 19 ionice -c 3 \  
    bash tests/observability/metrics_scrape_test.sh
```

Environment variables

Variable	Default	Meaning
RED_MODE	0	1 = RED reproduction, 0 = GREEN guard
HELIX_METRICS_URL	http:// 127.0.0.1:34090/ metrics	scrape URL (METRICS_PORT=34090)
HELIX_OBSERVABILITY_STACK	unset	set 1 (conductor, post-boot) to declare proxy-api up
HELIX_PROXY_URL	http:// 127.0.0.1:34128	HTTP proxy for the proxied-request step
HELIX_METRICS_PROBE_TARGET	http://target-a.internal/	URL fetched through the proxy
HELIX_METRICS_BYTEPATH_WIRED	0	set 1 once the byte-path to api counter increment lands
HELIX_METRICS_EVIDENCE_DIR	qa-results/ observability/ metrics_scrape/ <run-id>	evidence dir
HELIX_PROBE_TIMEOUT	10	curl --max-time for probes

Edge cases

- **Not booted, GREEN:** HELIX_OBSERVABILITY_STACK unset yields SKIP: topology_unsupported (authored-not-booted), return 0.
- **Declared up but /metrics dead:** HELIX_OBSERVABILITY_STACK=1 with no valid exposition is a FAIL — the metrics-not-exposed defect is live.
- **Counter flat after a proxied request:** today the byte-path to api counter increment is P5/P10-pending (control-plane/internal/api/metrics.go:14-17); the acl-helper writes OK/ERR to Squid's stdout and does NOT call the api process (control-plane/cmd/acl-helper/main.go). So a flat counter is an honest feature_disabled_by_config SKIP for the increment sub-proof, while the exposition-content proof still PASSes. Once the

wiring lands the conductor sets `HELIX_METRICS_BYTEPATH_WIRED=1`, after which a flat counter is a HARD FAIL (regression guard).

- **Proxy unreachable:** the increment sub-proof cannot be driven; it is skipped for that run while the exposition-content proof stands.

Internal behaviour

1. Resource cap: the script self-re-execs under `nice -n 19 ionice -c 3` with `GOMAXPROCS=2` when those tools are present.
2. Sources `tests/lib/evidence.sh`.
3. Scrapes `HELIX_METRICS_URL` (body + HTTP code captured to the evidence dir).
4. RED: asserts the scrape is NOT valid exposition (else FAIL — nothing to reproduce). GREEN: gates on `HELIX_OBSERVABILITY_STACK`, then asserts the content, then drives one proxied request and re-scrapes to compute the acl-decisions counter delta.
5. Emits structured PASS/FAIL/SKIP verdict lines and cites the captured scrape artefacts as evidence.

Wiring FACTS (file:line)

- Metrics-listener enable var `CONTROL_API_METRICS_ADDR` to `Config.MetricsAddr: control-plane/cmd/api/main.go:93`; documented `internal/api/api.go:19,31`; consumed `internal/api/server.go:134` (empty means off).
- Metrics path `/metrics` (GET): `internal/api/server.go:100` (mTLS mux) and `internal/api/server.go:112` (metricsRoutes plaintext mux).
- mTLS certs required before the listener starts (fail-closed): `internal/api/tls.go:26`; `startMetricsListener` runs after `buildTLSConfig` in `internal/api/server.go:170-175`.
- Metric names: `internal/api/metrics.go:35-37`.

Related scripts

- `docker-compose.observability.yml` — the compose overlay adding proxy-api with the metrics listener enabled.
- `tests/lib/evidence.sh` — the canonical anti-bluff evidence helper.
- `tests/dynamic/suites/ddos_flood_suite.sh` — the RED/GREEN + honest-SKIP pattern this guard follows.
- `config/prometheus/prometheus.yml` — the committed scrape config (job `helix-control-plane`, target `proxy-control-plane:34090`).

Conductor live-proof command sequence

The conductor (single owner of the live boot, §11.4.119) runs, from the repo root, after provisioning the four Podman secrets:

```
podman secret create helixproxy_pg_password - < /path/pg_password
podman secret create helixproxy_api_cert - < /path/server.crt
podman secret create helixproxy_api_key - < /path/server.key
podman secret create helixproxy_api_client_ca - < /path/client-
ca.crt
```

```
COMPOSE="podman-compose -f docker-compose.yml -f docker-
compose.dynamic.yml -f docker-compose.observability.yml --
profile dynamic"
```

```
$COMPOSE up -d proxy-postgres proxy-redis
```

```
$COMPOSE up -d proxy-api
```

```
HELIX_OBSERVABILITY_STACK=1 \
HELIX_METRICS_URL=http://127.0.0.1:34090/metrics \
HELIX_PROXY_URL=http://127.0.0.1:34128 \
GOMAXPROCS=2 nice -n 19 ionice -c 3 \
    bash tests/observability/metrics_scrape_test.sh
```

Last verified

- 2026-07-01 — authored; sh -n + bash -n clean; RED baseline PASS + GREEN authored-not-booted SKIP confirmed without booting; overlay merges clean under podman-compose config. Live GREEN scrape proof is owed to the conductor.